

Muddiest Points I – Sonstige Fragen

Frage: Ich habe das mit der Hilfsfunktion noch nicht verstanden. Wie kann ich mir erklären lassen, wofür ich eine Funktion gebrauchen kann?

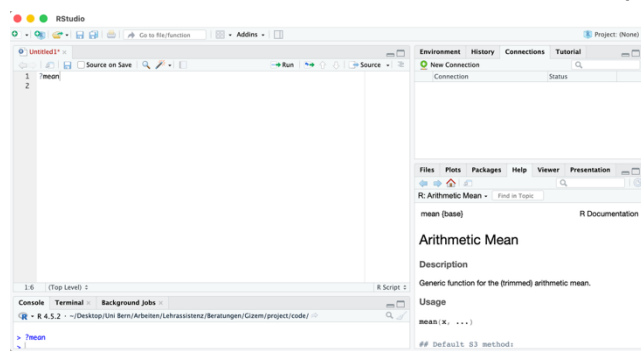
Antwort: In R gibt es eine eingebaute Hilfsfunktion, mit der du dir zu jeder Funktion eine ausführliche Beschreibung anzeigen lassen kannst.

Dafür gibt es zwei einfache Wege:

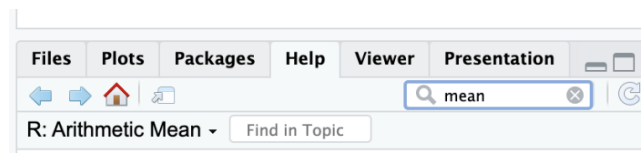
1. **Über die Konsole:** Du kannst in der Konsole ein Fragezeichen vor den entsprechenden Funktionsnamen setzen und diesen Code ausführen:

?mean

Dadurch öffnet sich automatisch die passende Hilfeseite im *Help*-Tab.



2. **Über den Help-Tab:** Du kannst auch direkt im *Help*-Fenster nach einer Funktion suchen.



Wie kannst du die Hilfeseite lesen?

Die Hilfeseiten sämtlicher Funktionen sind immer sehr ähnlich aufgebaut und enthalten mehrere wichtige Abschnitte:

mean (base)	← Aus welchem Paket stammt die Funktion?	R Documentation
Arithmetic Mean		
Description	← Kurze Erklärung, was die Funktion grundsätzlich macht	
Generic function for the (trimmed) arithmetic mean.		
Usage	← Zeigt, wie die Funktion verwendet wird (inkl. der Argumente)	
mean(x, ...)		
## Default S3 method:	← Zeigt die Default-Einstellungen der Argumente an (wenn diese nicht spezifisch anders durch den User angegeben werden)	
mean(x, trim = 0, na.rm = FALSE, ...)		
Arguments	← Erklärung der einzelnen Eingaben; was musst bzw. kannst du der Funktion übergeben?	
x	an R object. Currently there are methods for numeric/logical vectors and date , date-time and time interval objects. Complex vectors are allowed for trim = 0, only.	
trim	the fraction (0 to 0.5) of observations to be trimmed from each end of x before the mean is computed. Values of trim outside that range are taken as the nearest endpoint.	
na.rm	a logical evaluating to TRUE or FALSE indicating whether NA values should be stripped before the computation proceeds.	
...	further arguments passed to or from other methods.	

Value ← Beschreibt, was die Funktion zurückgibt

If `trim` is zero (the default), the arithmetic mean of the values in `x` is computed, as a numeric or complex vector of length one. If `x` is not logical (coerced to numeric), numeric (including integer) or complex, `NA_real_` is returned, with a warning.

If `trim` is non-zero, a symmetrically trimmed mean is computed with a fraction of `trim` observations deleted from each end before the mean is computed.

References

Becker, R. A., Chambers, J. M. and Wilks, A. R. (1988) *The New S Language*. Wadsworth & Brooks/Cole.

See Also ← Zeigt verwandte/ähnliche Funktionen an

[weighted.mean](#), [mean.POSIXct](#), [colMeans](#) for row and column means.

Examples ← Konkrete Beispiele, die selbst in der Konsole ausprobiert werden können; hilft meistens enorm viel für das Verständnis der Funktion

[Run examples](#)

```
x <- c(0:10, 50)
xm <- mean(x)
c(xm, mean(x, trim = 0.10))
```

[Package base version 4.5.2 [Index](#)]

Frage: Könnten wir noch automatisiertes Dateneinlesen behandeln?

Antwort: Wir werden das automatisierte Einlesen von mehreren Daten im Seminar leider nicht bearbeiten können, da dies zeitlich nicht reicht und auch zu anspruchsvoll wäre. Hier aber einige grundsätzliche Informationen:

Beim automatisierten Einlesen mehrerer Dateien kann grundsätzlich zwischen zwei Varianten unterschieden werden:

1. Alle Dateien einlesen und in einer gemeinsamen Liste speichern
2. Jede Datei als separates Objekt im Environment speichern

Variante 1: Mehrere Dateien automatisiert einlesen und in einer Liste speichern

Gehen wir davon aus, dass wir einen Ordner namens `data` haben, welcher mehrere CSV-Dateien beinhaltet. Diese wollen wir nun alle in einem Schritt einlesen und in einer übergeordneten Liste abspeichern:

```
1 # 1) Alle CSV-Dateien im Ordner "data/" auflisten
2 files <- list.files(
3   path = "data/",
4   pattern = "\\*.csv$",
5   full.names = TRUE
6 )
7
8 # 2) Zur Kontrolle anzeigen, welche Dateien gefunden wurden
9 files
10
11 # 3) Alle Dateien einlesen und in einer Liste speichern
12 data_list <- lapply(files, read.csv)
13
14 # 4) Den Listenelementen sinnvolle Namen geben
15 names(data_list) <- tools::file_path_sans_ext(basename(files))
16
17 # 5) Liste anschauen
18 data_list
19
20
```

Diese Funktion sucht nach Dateien in einem Ordner:

- `path = "data/"` gibt an, in welchem Ordner gesucht werden soll
- `pattern = "\\*.csv$"` sorgt dafür, dass nur CSV-Dateien gefunden werden (dies kann entsprechend angepasst werden)
- `full.names = TRUE` gibt den vollständigen Pfad zurück

Damit kannst du prüfen, ob wirklich die richtigen Dateien gefunden wurden.

`lapply()` führt dieselbe Funktion für jedes Element eines Vektors aus. Hier wird also jede Datei im Objekt `files` genommen und darauf `read.csv()` angewendet. Das alles wird in einer Liste namens `data_list` gespeichert.

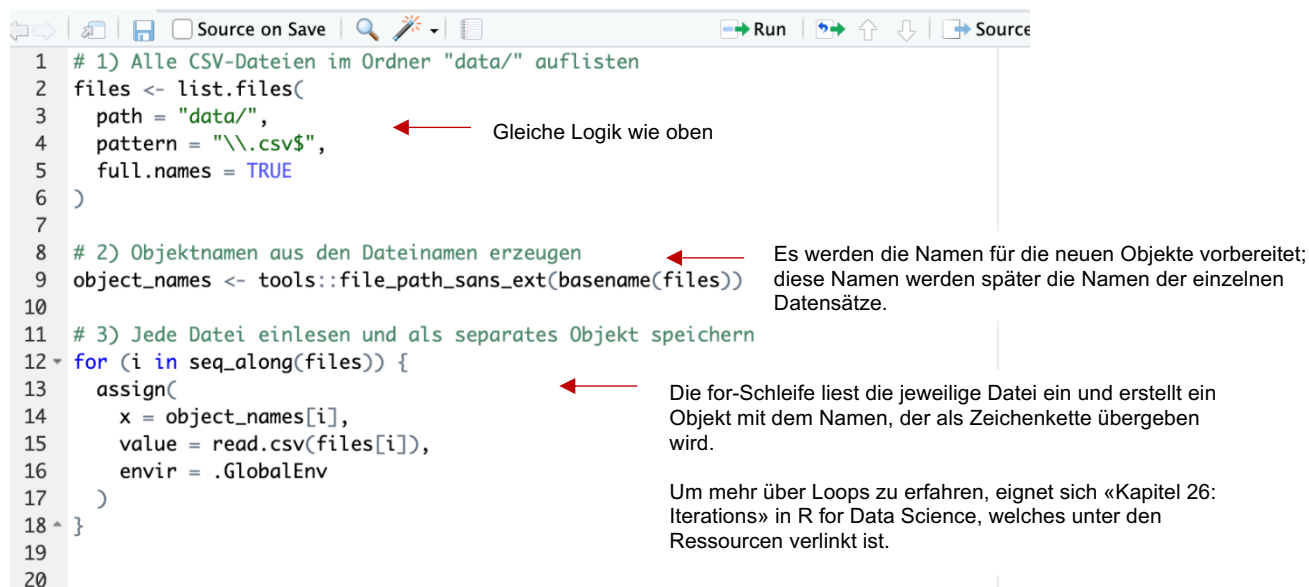
Ohne diesen Schritt würden die Elemente nur `[[1]]`, `[[2]]` usw. heißen. So erhalten sie sinnvolle Namen.

- `basename(files)` entfernt den Ordnerpfad und lässt nur den Dateinamen übrig.
- `tools::file_path_sans_ext(...)` entfernt die Dateierweiterung `.csv`

Es kann dann auf einzelne Datensätze in der Liste zugegriffen werden (z.B. `data_list$studie1` oder `data_list[[1]]`).

Variante 2: Mehrere Dateien einlesen und als separate Datensätze speichern

Wir können auch jede Datei als eigenes Objekt im Global Environment anlegen.



```
1 # 1) Alle CSV-Dateien im Ordner "data/" auflisten
2 files <- list.files(
3   path = "data/",
4   pattern = "\\*.csv$",
5   full.names = TRUE
6 )
7
8 # 2) Objektnamen aus den Dateinamen erzeugen
9 object_names <- tools::file_path_sans_ext(basename(files))
10
11 # 3) Jede Datei einlesen und als separates Objekt speichern
12 for (i in seq_along(files)) {
13   assign(
14     x = object_names[i],
15     value = read.csv(files[i]),
16     envir = .GlobalEnv
17   )
18 }
19
20
```

Gleiche Logik wie oben

Es werden die Namen für die neuen Objekte vorbereitet; diese Namen werden später die Namen der einzelnen Datensätze.

Die for-Schleife liest die jeweilige Datei ein und erstellt ein Objekt mit dem Namen, der als Zeichenkette übergeben wird.

Um mehr über Loops zu erfahren, eignet sich «Kapitel 26: Iterations» in R for Data Science, welches unter den Ressourcen verlinkt ist.

Es kann danach direkt wie gewohnt mit den einzelnen Datensätzen gearbeitet werden.

Frage: Die Thematik mit den LLMs ging mir ein bisschen zu schnell, aber soweit ich dies verstanden habe, ist es nur eine Möglichkeit, diese anzuwenden und keine Pflicht.

Antwort: Die Nutzung von LLMs (z.B. KI-Tools in R) ist im Seminar selbstverständlich keine Pflicht. Wir haben diese Thematik aufgegriffen, um einen Einblick in mögliche Anwendungen zu geben und zu zeigen, wie solche Tools direkt in R eingesetzt werden können, etwa zur Unterstützung beim Schreiben oder Verstehen von Code. Die gezeigten Beispiele dienen dabei ausschliesslich der Demonstration. Entscheidend ist, dass die Inhalte verstanden und eigenständig angewendet werden können; unabhängig davon, ob solche Tools genutzt werden oder nicht.

Informationen zur Integration und Installation der verschiedenen Tools in R-Studio findest du auf der Webseite unter den [Links und Ressourcen](#).

Frage: Ich bin mir noch nicht ganz sicher, ob ich das Mergen verstanden habe. Evtl. dort noch einmal aufzeigen, wofür das genau ist?

Antwort: Das Zusammenführen (Mergen) von Datensätzen ist ein zentraler Schritt in der Datenanalyse, da relevante Informationen oft nicht in einem einzigen Datensatz vorliegen,

sondern auf mehrere Dateien verteilt sind. Ziel des Mergens ist es, diese Informationen sinnvoll zu kombinieren, sodass alle benötigten Variablen gemeinsam analysiert werden können. Ein typisches Beispiel ist, dass in einem Datensatz z.B. Leistungsdaten von Studierenden enthalten sind, während ein anderer Datensatz zusätzliche Informationen wie Alter oder Geschlecht beinhaltet. Erst wenn diese beiden Datensätze zusammengeführt werden, können Zusammenhänge zwischen diesen Variablen untersucht werden.

In [Block 2](#) der Übungen haben wir diesen grundlegenden Zweck bereits besprochen und auch zwei zentrale Methoden kennengelernt: `cbind()` und `full_join()`. Dabei ist es wichtig zu verstehen, dass beide Funktionen Datensätze unterschiedlich zusammenführen:

- `cbind()` fügt Datensätze rein spaltenweise zusammen und setzt voraus, dass die Reihenfolge der Beobachtungen in beiden Datensätzen exakt übereinstimmt.
- `full_join()` hingegen nutzt eine sogenannte Schlüsselvariable (z.B. eine ID), um die passenden Zeilen korrekt zueinander zuzuordnen. Dadurch ist diese Methode in der Praxis robuster und zu empfehlen, insbesondere wenn die Reihenfolge der Daten nicht identisch ist oder Beobachtungen fehlen.

Insgesamt dient das Mergen also dazu, verteilte Informationen zusammenzuführen und damit die Grundlage für weiterführende Analysen zu schaffen.

Frage: *Strukturierte Ordner: Gute Organisation von R-Projekten und richtiges Abspeichern von Daten und Skripten.*

Antwort: Eine strukturierte Organisation von Ordnern, Daten und Skripten ist ein zentraler Bestandteil guter wissenschaftlicher Praxis und besonders wichtig bei der Arbeit mit R-Projekten. Der Hauptgrund dafür ist, dass Datenanalysen in der Regel aus vielen einzelnen Schritten bestehen: Daten werden eingelesen, bereinigt, analysiert und Ergebnisse werden gespeichert. Ohne eine klare Struktur wird es schnell schwierig nachzuvollziehen, wo sich welche Daten befinden und welche Version aktuell ist.

Eine gute Ordnerstruktur hilft dabei, diese einzelnen Bestandteile klar zu trennen. Typischerweise werden beispielsweise Rohdaten, aufbereitete Daten, Skripte und Ergebnisse in unterschiedlichen Ordnern gespeichert. Dadurch ist sofort ersichtlich, welche Dateien wofür verwendet werden, und es wird verhindert, dass Daten versehentlich überschrieben oder verändert werden. Gleichzeitig erleichtert dies das gezielte Arbeiten, da benötigte Dateien schnell gefunden werden können.

Ein weiterer zentraler Vorteil ist die Reproduzierbarkeit: Wenn Daten und Skripte sauber organisiert sind, können Analysen jederzeit nachvollzogen und erneut ausgeführt werden; auch von anderen Personen.

Für die praktische Umsetzung bedeutet das, dass man von Beginn an eine klare Struktur im Projekt anlegt, konsistente Dateinamen verwende und alle relevanten Dateien innerhalb eines R-Projektes organisiert. Dadurch kann auch mit relativen Pfaden gearbeitet werden, was den Code robuster und leichter übertragbar macht.

👉 Einen konkreten Vorschlag für eine solche Struktur sowie weiterführende Hinweise haben wir in den Folien von Einheit 3 (EH3) besprochen. Dort wird insbesondere auch der Standard PsychDS vorgestellt, an dem wir uns im Seminar orientieren.

Frage: *Das ist jetzt nicht direkt 1 zu 1 lernbares, aber ich habe z.T. noch Mühe zu kontrollieren, ob meine Codes wirklich das gemacht haben, was ich wollte. Also wenn keine Fehlermeldung kommt, der Code ausgeführt wird (z.B. Datensätze mergen) und ich dann kontrollieren will, ob ich nun auch wirklich das gemacht habe, das ich tun sollte oder den falschen Code genommen habe. Da stolpere ich z.T. noch und frage mich, ob es irgendwelche allgemeinen Tricks gibt, um Outputs, resp. Datensätze auf Plausibilität zu überprüfen?*

Antwort: Dass Code ohne Fehlermeldung läuft, bedeutet leider nicht automatisch, dass er auch das gemacht hat, was beabsichtigt war. Gerade bei Operationen wie dem Mergen von Datensätzen ist es daher sehr wichtig, die Ergebnisse aktiv auf Plausibilität zu überprüfen. Diese Form der Kontrolle ist ein zentraler Bestandteil guter Datenanalyse.

Ein wichtiger erster Schritt beginnt dabei bereits vor dem Ausführen des Codes: Es ist sinnvoll, sich bewusst zu überlegen, was genau man erwartet. Wie sollte der Datensatz nach der Funktion bzw. dem Befehl aussehen? Wie viele Beobachtungen sollten vorhanden sein? Welche Variablen sollten neu hinzukommen? Code sollte also nicht «einfach ausprobiert» werden, sondern immer mit einer klaren Erwartung verbunden sein, die anschliessend überprüft wird.

Nach der Ausführung kann man den Datensatz gezielt inspizieren, z.B. mit Funktionen wie `head()`, `View()` oder `str()`. So lässt sich schnell überprüfen, ob die erwarteten Variablen vorhanden sind und ob die Struktur plausibel ist. Ebenso ist es hilfreich, die Anzahl der Beobachtungen vor und nach einer Operation zu vergleichen (z.B. mit `nrow()`), um sicherzustellen, dass keine unerwarteten Fälle verloren gegangen oder hinzugekommen sind.

Gerade beim Mergen ist es sinnvoll zu prüfen, ob die Zusammenführung korrekt funktioniert hat. Dazu kann man beispielsweise kontrollieren, ob die Schlüsselvariable wie erwartet übereinstimmt oder ob ungewöhnlich viele fehlende Werte (NA) entstanden sind, was auf Probleme beim Zusammenführen hindeuten kann.

Zusätzlich kann es hilfreich sein, Code in ein LLM (z.B. ein KI-Tool) einzufügen und sich erklären zu lassen, was genau dieser Code macht. Diese Erklärung kann dann mit den eigenen Erwartungen abgeglichen werden. Stimmen beide überein, ist das ein gutes Zeichen; gibt es Abweichungen, lohnt es sich, den Code nochmals kritisch zu prüfen.

Allgemein gilt: Ergebnisse sollten immer auch inhaltlich hinterfragt werden. Wirkt das Ergebnis logisch? Passen die Werte zu dem, was man erwartet hätte? Ein kurzer Blick auf Verteilungen oder einfache Zusammenfassungen (z.B. mit `summary()`) kann oft schon Hinweise auf mögliche Fehler geben. Wenn man Berechnungen durchführt (z.B. Mittelwert über Fragebogenitems), kann es auch helfen dies für einzelne Versuchspersonen händisch nachzurechnen, um zu sehen ob es richtig durchgeführt wurde.

Frage: *Beim Importieren von Daten ist mir folgendes noch nicht klar: darf ich beim Importieren eines datasets rechts unten im Skript auf file gehen, dann das dataset anklicken, dann auf «import dataset» klicken, den Code kopieren und Einfügen und dann*

diesen Code ausführen? Weil ihr habt es über das Environment – import dataset gezeigt und das funktioniert bei mir nicht immer. Aber so wie ich es mache, hat es immer funktioniert. Ist es wichtig, dass man den Code vom Environment nimmt?

Antwort: Grundsätzlich ist es völlig in Ordnung, wenn du den Weg über «File -> Import Dataset» gehst, den generierten Code kopierst, in dein Skript einfügst und dann ausführst. Wichtig ist dabei vor allem, dass du den Code tatsächlich in deinem Skript speicherst und nicht nur über die Benutzeroberfläche arbeitest. Denn nur so bleibt dein Vorgehen nachvollziehbar und reproduzierbar.

Der Weg über das Environment («Import Dataset») ist also keine Pflicht, sondern lediglich eine alternative Möglichkeit, um sich den passenden Import-Code automatisch generieren zu lassen. Wenn dieser Weg bei dir nicht zuverlässig funktioniert, ist es absolut legitim, den Import über das Menu zu machen und den Code zu übernehmen.

Entscheidend ist nicht, wie du den Code erzeugst, sondern dass du:

- den Code im Skript hast
- ihn verstehst
- und ihn jederzeit wieder ausführen kannst

Langfristig ist es jedoch sinnvoll, sich mit den Importfunktionen (z.B. `read_csv()` oder `read_excel()`) direkt vertraut zu machen und diese selbst zu schreiben. So wirst du unabhängiger von der Benutzeroberfläche und arbeitest effizienter.